TD2 - Communication on a ring

Organization. The exercises that we cannot cover in class are homework: their solutions will be posted, and we may go over some of them later in class. The algorithms of an exercise may be used in the following exercises.

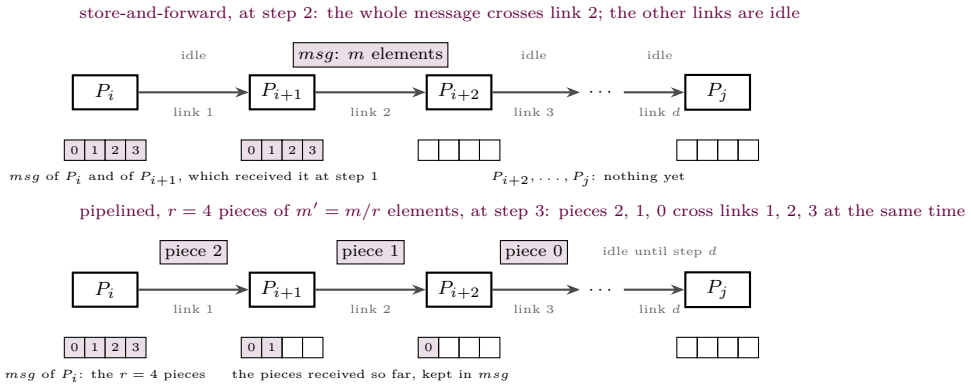
Conventions. The machine is the unidirectional ring of the lecture: p processors $P_0, \dots, P_{p-1}$, each with its own memory; P_k has one outgoing link, to P_{k+1}, and one incoming link, from P_{k-1} (indices modulo p). Every processor runs the same program, with four routines: MyRank() and NumProcs() return the rank k of the processor and p; Send(msg, m) sends to P_{k+1} the m elements of the array starting at msg; Recv(msg, m) receives m elements from P_{k-1} and stores them from msg on. Arrays are passed by name, with their size as a separate parameter (m, n), and p is always NumProcs(); $msg[a \dots]$ as an argument denotes the array starting at its element a (the elements $msg[a], msg[a+1], \dots$), and $M[j]$ the row j of a 2-D array M, an array like any other; an assignment between arrays copies them. Send does not block the processor, Recv blocks it until the message has arrived, and a processor can send and receive at the same time: Send$(\dots) \parallel$ Recv$(\dots)$. A message of m elements costs $L + m\,b$ on a link (L: latency, b: time per element); messages on different links travel at the same time, messages on the same link one after the other; an arithmetic operation costs τ; in practice $\tau \le b \ll L$: transferring an element costs at least as much as operating on it, typically 10 to 100 times more, and the latency is larger by three orders of magnitude again (τ, b: nanoseconds; L: microseconds). The cost of an algorithm is the time until the last processor is done; we distinguish its communication part T_{comm} and its computation part T_{comp}. The following routines of the lecture may be called as building blocks, and so may their pipelined versions of Exercises 2 and 3, which have the same signatures, once they have been established; their costs, with and without pipelining, are the object of Exercises 2 and 3.

Algorithm	Effect
Bcast(k, msg, m)	copies the array msg of P_k into the array msg of every other processor
Reduce$(k, msg, m, \otimes)$	P_k gets in msg the element-wise $\otimes$ of the arrays msg of all the processors; the others keep theirs

Exercise 1

Pipelined point-to-point communication

Processor P_i must send a message of m elements, stored in its array msg, to processor P_j ($j \neq i$); on the ring, the message has to go through $P_{i+1}, \dots, P_{j-1}$ (indices modulo p: the message may wrap around the ring) and crosses $d = (j - i) \bmod p$ links. In the *store-and-forward* protocol, every intermediate processor receives the whole message, then forwards it, which costs $c_{i,j}(m) = d\,(L + m\,b)$: a single link is busy at a time, while the $d - 1$ others are idle. To use them, we *pipeline* the transfer: the message is cut into r pieces of $m' = m/r$ elements (assume that r divides m), sent one after the other; every intermediate processor forwards the piece it received at the previous step while it receives the next one.



a) Write the algorithm SendTo(i, j, msg, m, r), executed by every processor, that performs this pipelined transfer; the processors outside $P_i, \dots, P_j$ do nothing, and P_j receives the message in its own array msg.

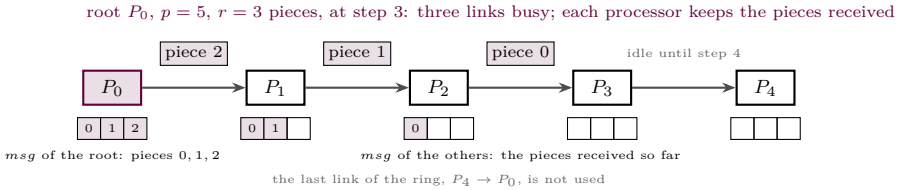
b) Find the cost $c_{i,j}(m, r)$ of the pipelined transfer, in terms of r, d, L, m and b.

c) Which value of r minimizes this cost? What is the cost then, and what does it become for large messages? Compare with the store-and-forward cost.

Exercise 2

Pipelined broadcast on a ring

The BCAST(k, msg, m) of the lecture relays the whole array of the root around the ring, in $(p-1)(L+m\,b)$: again a single link is busy at a time. Pipelining applies: the root cuts its array into r pieces of m/r elements (r divides m) and sends them one after the other; every other processor forwards each piece as soon as it has it, while it receives the next one, and keeps a copy. The number of pieces r is not a parameter: the algorithm chooses it internally, as well as possible (question (c)).



a) Write the pipelined algorithm BCAST(k, msg, m), with the signature of the lecture.

b) Find its cost in terms of p, L, m, b and the number of pieces r.

c) Which value of r minimizes this cost, and is therefore the one the algorithm uses? What is the cost then, and what does it become for large messages? Compare with the $(p-1)(L+m\,b)$ of the lecture.

Exercise 3

Reduction on a ring

REDUCE$(k, msg, m, \otimes)$: every processor holds m elements in its array msg; at the end, the root P_k holds in msg their combination, element by element, with the associative and commutative operation $\otimes$ (sum, max, ...). The lecture gave its principle: a partial result travels around the ring from P_{k+1} to P_k, every processor combining what it receives with its own elements before forwarding it. Combining s received elements, held in a buffer tmp, into s elements of msg is a loop $msg[a+l] \leftarrow tmp[l] \otimes msg[a+l]$, $l = 0, \ldots, s-1$, which costs $s\,\tau$.

a) Write the non-pipelined algorithm REDUCE$(k, msg, m, \otimes)$, with the structure of the BCAST of the lecture.

b) Find its communication cost T_{comm} and its computation cost T_{comp}.

c) Pipeline it with r pieces of m/r elements, the number of pieces being chosen internally by the algorithm (question (e)): write the pipelined REDUCE$(k, msg, m, \otimes)$, with the same signature. Be careful: a piece can be forwarded only once it has been received *and* combined.

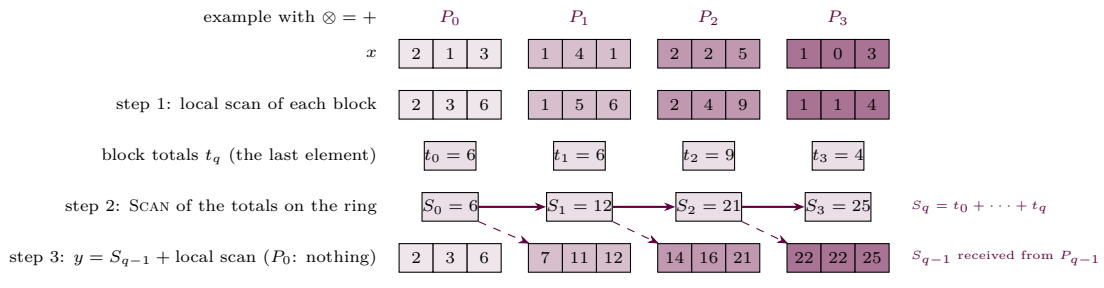
d) Find its costs T_{comm} and T_{comp} (draw what each processor does over time).

e) Which value of r minimizes the total cost? This is the value the algorithm uses.

Exercise 4

Scan on a ring

An array x of N elements is distributed by blocks of $n = N/p$ (p divides N): P_q holds the elements $x[q\,n], \ldots, x[(q+1)n-1]$ in a local array $x[0 \ldots n-1]$. We want its *scan* for an associative operation $\otimes$ (sum, max, ...), as in the PRAM session: $y[i] = x[0] \otimes x[1] \otimes \cdots \otimes x[i]$ for $i = 0, \ldots, N-1$, distributed in the same way: P_q must compute $y[q\,n], \ldots, y[(q+1)n-1]$ in a local array $y[0 \ldots n-1]$. We follow the SCANP of the PRAM session: every processor scans its own block; the p block totals are scanned across the processors; every processor then finishes its block with the result.



a) Write Scan$(v, \otimes)$ on the ring: every processor holds one value v; at the end, P_q holds $v_0 \otimes v_1 \otimes \cdots \otimes v_q$ in v (v_i: the initial value of P_i). Find its costs T_{comm} and T_{comp}. Can it be pipelined like Bcast? Can any algorithm do better on the unidirectional ring?

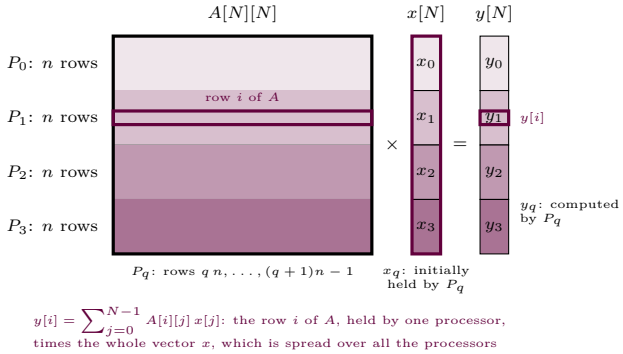
b) Write ScanP$(x, y, n, \otimes)$, which first scans the local block, then calls Scan on the block totals, and finally finishes the block with a last update. Which quantity does this update combine with the elements of P_q, and how does P_q obtain it? Find the costs T_{comm} and T_{comp} of ScanP.

c) The sequential scan costs $(N-1)\,\tau \approx N\,\tau$. What are the speedup and the efficiency of ScanP? What is the limit of the efficiency for large N, and why is it not 1?

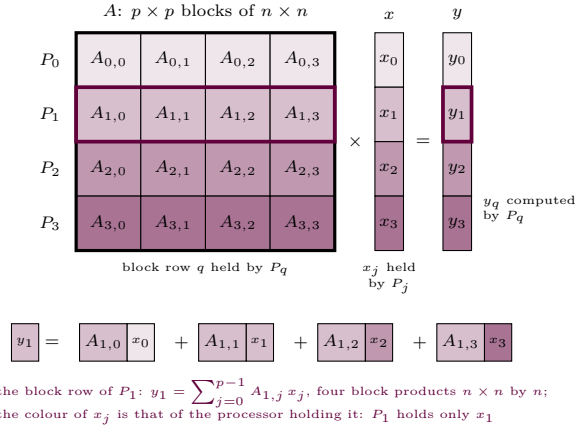
Exercise 5

Matrix–vector product on a ring

Let A be an $N \times N$ matrix and x a vector of N elements; we want to compute $y = A\,x$, that is $y[i] = \sum_{j=0}^{N-1} A[i][j]\,x[j]$. The data are distributed by blocks of $n = N/p$ (p divides N): P_q holds the rows $q\,n, \ldots, (q+1)n-1$ of A in a local array $A[0 \ldots n-1][0 \ldots N-1]$, and the elements $x[q\,n], \ldots, x[(q+1)n-1]$ of x in a local array $x[0 \ldots n-1]$; it must compute the elements $y[q\,n], \ldots, y[(q+1)n-1]$ of y in a local array $y[0 \ldots n-1]$.



Seen by blocks: cut A into $p \times p$ blocks of $n \times n$ elements, $A_{q,j}$ being the block at the rows of P_q and the columns $j\,n, \ldots, (j+1)n-1$, and cut x and y into p blocks of n elements, x_j and y_j. Then $y_q = \sum_{j=0}^{p-1} A_{q,j}\,x_j$: the block y_q of P_q is the sum of p small matrix–vector products, one per block of x, each one an $n \times n$ block times n elements. P_q holds the whole block row $A_{q,0}, \ldots, A_{q,p-1}$ of A, but a single block of x, its own x_q.



a) Which elements of x does P_q need to compute its block of y? Which processors hold them? How many phases of computation and of communication must take place, and what should be computed and communicated in each phase?

b) Design MatVec(A, x, y, n) on the ring, where A is the local array of n rows and $N = n\,p$ columns of the processor, and x and y its local arrays of n elements: the blocks of x travel around the ring; at every step, every processor uses the block it holds, then passes it to its right neighbour.

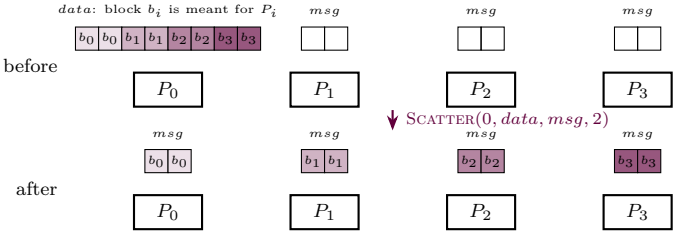
c) Find its communication cost and its computation cost.

d) The sequential product costs $2N^2\,\tau$. What are the speedup and the efficiency of MatVec? When is the efficiency close to 1?

Exercise 6

Scatter on a ring

In many applications, the data are initially held by a single processor (the one that read them from a file, say) and must be distributed in equal parts to all the processors, instead of starting from an already distributed configuration. Scatter$(k, data, msg, m)$ does exactly this: the root P_k holds an array $data$ of pm elements, made of p consecutive blocks of m elements, the block $b_i = data[i\,m \dots (i+1)m-1]$ being meant for P_i; at the end, every P_i holds its block b_i in its array msg of m elements.



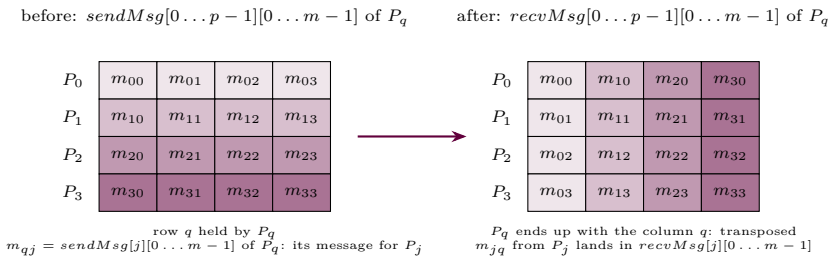
a) Design Scatter$(k, data, msg, m)$ on the ring. Start with the naive version: the root keeps its block and sends all the others to its right neighbour, which keeps its block and sends the rest on, and so on. Then make the whole operation a pipeline: in which order should the root send the blocks, so that no block waits behind another one? Finally, write a third version made of $p-1$ calls to the SendTo of Exercise 1.

b) Find the communication costs of the three versions. Explain why the cost of the pipelined version is the same as that of Bcast although $p-1$ different blocks are sent, and when the version based on SendTo is as good.

Exercise 7

Transposition on a ring

Every processor P_q holds p messages of m elements in a $p \times m$ array $sendMsg[0 \dots p-1][0 \dots m-1]$: the row $sendMsg[j][0 \dots m-1]$ is the message meant for P_j ($sendMsg[q][0 \dots m-1]$ for itself). At the end, every P_q must hold in the row $recvMsg[j][0 \dots m-1]$ of a $p \times m$ array $recvMsg$ the message that P_j sent to it, for every j. Seen as a $p \times p$ array of messages whose row q is held by P_q, the operation transposes the array: P_q ends up with the column q. This exchange is the *all-to-all* of MPI.



a) Design Transposition$(sendMsg, recvMsg, m)$ on the ring, in two steps. First write Shift(src, dst, m, k): every P_q sends the m elements of its array src to $P_{(q+k) \bmod p}$ and receives in dst those of $P_{(q-k) \bmod p}$, a cyclic shift of the messages by k positions. Then use it: proceed in $p-1$ rounds; in round s, every processor sends the message meant for the processor at distance s.

b) Find its communication cost in terms of p, L, m and b.

c) Count the total number of link crossings that any algorithm must perform, and deduce a lower bound on the cost of a transposition on the ring. Is your algorithm optimal?

Exercise 8

Grids, hypercubes and binary trees

Back to the parameters of the lecture, on three other topologies.

a) What is the diameter of a complete binary tree of height h (the root at depth 0, the leaves at depth h)?

b) On an $n \times n$ grid, how many different shortest paths are there between the processors at positions (x_1, y_1) and (x_2, y_2)?

c) Prove that the bisection width of the d-cube ($p = 2^d$ processors) is 2^{d-1}: exhibit a cut of 2^{d-1} links that splits it into two halves, then show that no cut with fewer links does.